1- During the creation or modification of a job offer, the client application legitimately submits a request setting the vacancy status to MODERATION. An attacker can intercept this GraphQL mutation and maliciously modify the status variable to ACTIVE. Because the backend server blindly trusts the client-provided input without enforcing Role-Based Access Control (RBAC) checks on the allowed state transitions, the job offer is immediately published and visible to all users.

2- An attacker acting as a restricted staff member (e.g., possessing only "Managed Store" permissions) can directly query the /signup_object/dev_store API endpoint. Because the endpoint only verifies general store access rather than the specific "Development Store" permission, it leaks a valid creation token. The attacker then submits this token to the final /services/signup/create POST endpoint. The backend processes the request, creates the development store, and automatically authenticates the attacker into the newly created, unauthorized environment.

3- • **Information Disclosure:** The WordPress REST API improperly exposes sensitive user data (emails) instead of restricting this information strictly to authenticated administrators.
• **CORS Bypass:** The endpoint is configured with permissive CORS headers, specifically `Access-Control-Allow-Credentials: true`, coupled with a dynamically reflected `Access-Control-Allow-Origin` header.
• An attacker can host a malicious HTML page containing an asynchronous JavaScript payload. When a victim visits this page, the browser sends an authenticated cross-origin request to the vulnerable endpoint. The permissive CORS policy allows the malicious script to read the sensitive JSON response and exfiltrate the victim's data to an attacker-controlled server.

4- First-party API keys are notoriously extractable from compiled client applications and have been publicly leaked. An attacker can integrate these leaked keys into a malicious third-party application. When a victim initiates the OAuth flow, the authorization screen falsely assures them that their DMs are safe. However, upon authorization, the attacker receives an Access Token endowed with the elevated privileges of the official first-party application, allowing complete and unauthorized read access to the victim's private DMs.

5- • **Access Control Bypass:** An attacker with a "Basic" role in Jira navigates directly to the app's configuration URL (`/plugins/servlet/ac/com.hackerone/get-started-with-hackerone-on-jira`).
• **Token Extraction:** Because the plugin does not verify if the requesting user possesses administrative privileges, it renders the configuration page and generates a valid linking URL containing a sensitive JWT.
• **Unauthorized Integration:** The attacker extracts the JWT and uses it to

claim the Jira application from their own, attacker-controlled HackerOne account.

6- By intercepting the POST request and modifying the `thread_id` parameter to target other existing conversations, an attacker can forcefully inject arbitrary messages into private threads they do not belong to. Because the `thread_id` values are sequential, an attacker can easily automate this process to iterate through all thread IDs, effectively spamming every private conversation on the platform.

7- The backend server correctly verifies user authentication but completely lacks object-level authorization checks. An attacker can simply intercept their own API requests and replace their conversation-id with the ID of a target victim (including administrators). By utilizing standard HTTP methods (GET, PATCH, DELETE), the attacker can illicitly read private chat histories, modify conversation metadata, or permanently delete the victim's threads without requiring any user interaction from the victim.

8- The researcher demonstrated the exploit by simply piping text into a network connection (echo [command] | ncat [IP] 2181):
envi (Environment): Leaked sensitive internal paths (/usr/src/zookeeper-locutus), Java versions, and hostnames.
dump: Leaked internal IP addresses (10.92.x.x) of the microservices (agents) connected to the cluster, mapping out the internal network.
stat: Showed active connections and performance metrics.
The "Nuke" Option (kill): The researcher correctly noted that they *could* have sent the kill command. If executed, this would immediately shut down the ZooKeeper node, potentially causing a massive Denial of Service (DoS) across Shopify's internal distributed architecture.

9- • **Direct Endpoint Access (Forced Browsing):** The attacker bypasses standard application authentication by directly navigating to a hidden, unprotected APEX User Management endpoint (`/ords/f?p=842:9`).
• **Rogue Account Provisioning:** Because the page lacks authentication checks, the unauthenticated attacker exploits the exposed "Add New User" form to create a legitimate, backend-recognized account using their own email address.
• **Vertical Privilege Escalation:** Utilizing the "Assign User Roles" feature on the same unprotected page, the attacker arbitrarily elevates the privileges of their newly created rogue account to "Administrator".
• **Full System Compromise:** The attacker retrieves the generated credentials from their email, navigates to the main application portal (`/ords/f?p=303`), and logs in. This results in a complete system takeover, granting the attacker full administrative access to view, modify, or delete restricted USG data and manage other users.

10- • The low-privileged attacker places a known malicious string inside a user-controlled directory. The highly privileged Acronis AV service detects it, moves it to quarantine, and leaves a 0-byte hidden "stub" file in the original location.

• **Environment Manipulation:** The attacker deletes the user-controlled folder containing the hidden stub file to clear the original file path.

• **Symlink Creation:** Utilizing Windows symbolic link techniques (creating a Mount Point to `\RPC Control\` and an object symlink), the attacker maps the original file's path to point directly to a restricted, highly privileged system file

• **Privileged Execution:** The attacker navigates to the Acronis AV GUI and executes the "Delete from PC" action for the quarantined file.

• **System Compromise:** The Acronis service, running with `SYSTEM` privileges, attempts to delete the original stub file without dropping privileges (impersonation) or checking for reparse points. It blindly follows the attacker-crafted symlink, successfully deleting the protected OS file.